

1. Problem Overview

1.1 Assigned Problem Statement

The assigned problem, MediSync AI: Intelligent Hospital Resource and Patient Flow Management System, addresses the challenge hospitals face in coordinating beds, staff, and equipment across departments. Traditional hospital information systems handle administrative records but generally lack intelligent decision-support for optimizing patient flow, which leads to overcrowding, uneven resource distribution, and longer waiting times. The brief calls for an AI-powered platform that continuously monitors resource availability, predicts admission and discharge demand using machine learning, and recommends efficient allocation of beds, staff, and equipment.

1.2 Implemented Solution Summary

The submitted repository (medsync-ai) implements the front-facing layer of this brief: a single-page web dashboard served by a lightweight Flask backend, packaged with Docker. The dashboard presents four summary cards (total beds, available beds, patients today, medical staff), a per-department patient flow table (Emergency, Outpatient, Laboratory, Inpatient with patient counts and a status label), and an "AI Prediction" panel that currently displays a static demand forecast and recommended action.

At this stage, the implementation is a UI/API scaffold rather than the full platform described in the problem statement — the values shown are hard-coded in the HTML template rather than computed from live hospital data or a trained model, and there is no persistence layer, admission/discharge prediction logic, or scheduling optimization yet. This gap between the problem statement's scope and the current build is addressed directly in Sections 3 and 7.

1.3 Key Functionalities Present in the Code

- A Flask route (/) that renders the dashboard template.
- A Flask health-check route (/health) that returns a JSON status payload — useful for container orchestration or uptime checks.

- A static dashboard UI (HTML/CSS, no JavaScript) showing bed occupancy, patient counts, staff counts, and a per-department flow table.
- A placeholder "AI Prediction" section intended to surface predictive analytics output.
- A Dockerfile that containerizes the Flask app for deployment.

2. Repository Organization

2.1 Repository Structure

The repository follows a minimal but understandable layout:

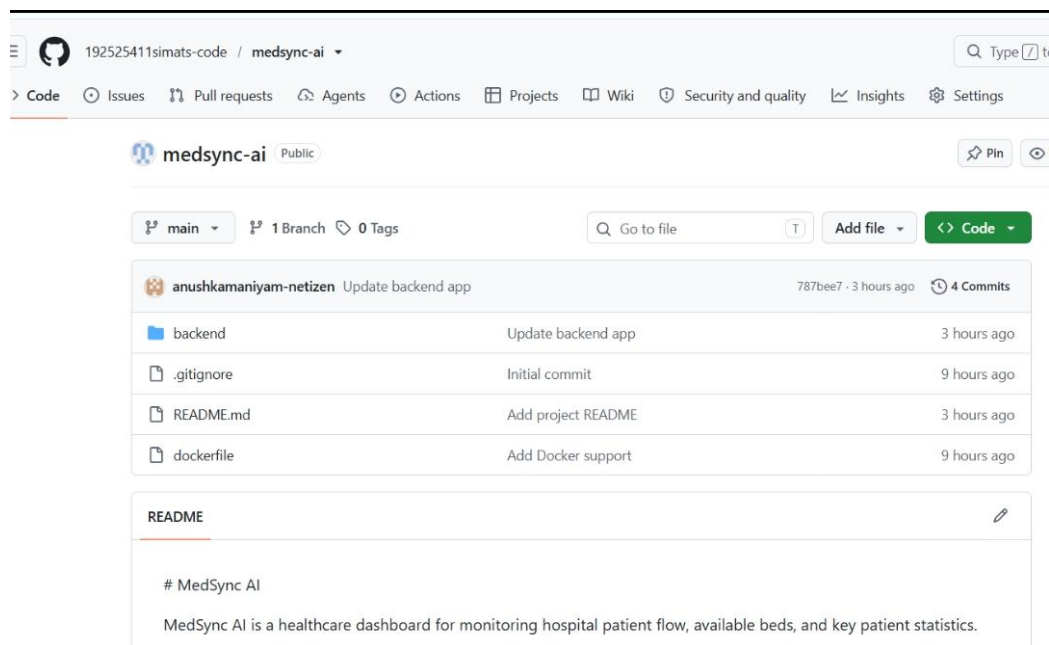
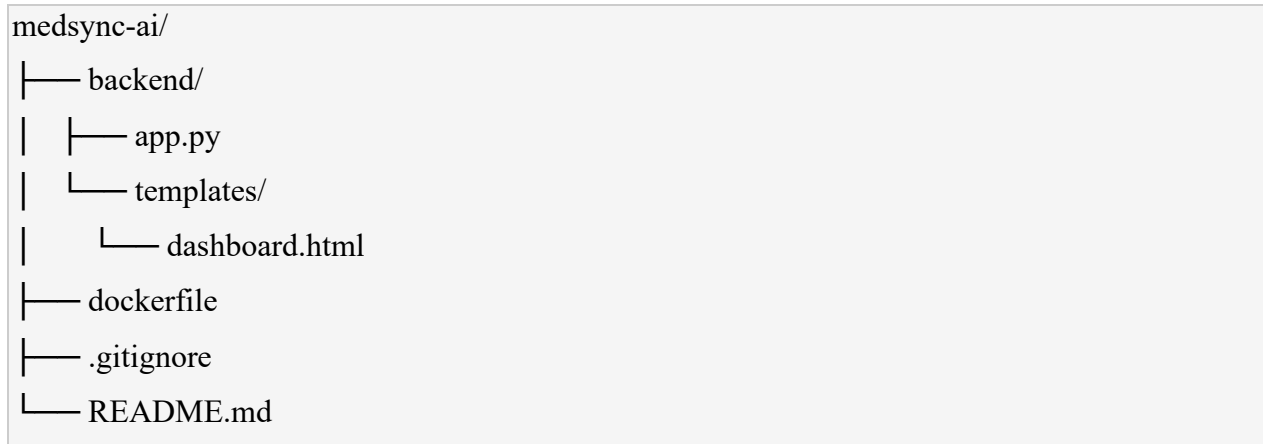


Fig. 1 — GitHub "Code" tab showing the repository file tree (main branch, 4 commits).

2.2 Evaluation

- A backend/ folder separates server-side code from the project root, and templates/ correctly follows Flask's default convention for HTML views — a reviewer familiar with Flask can locate the entry point immediately.
- File naming (app.py, dashboard.html, dockerfile, README.md) is conventional and lower-case, consistent with common Python/Flask practice.
- The structure does not yet separate concerns that will matter as the project grows toward the full problem statement: there is no static/ folder for CSS/JS (currently inlined in the HTML), no models/ or ml/ directory for the prediction component, no config or requirements file, and no tests/ directory.
- The Dockerfile sits at the repository root rather than alongside the backend it packages, which is a reasonable and common choice, but it should be accompanied by a docker-compose.yml once a database or additional services are introduced.

2.3 Justification for Maintainability and Collaboration

For a two-file backend, the current layout is easy to navigate and a new contributor could orient themselves within minutes. However, it will not scale cleanly once prediction logic, a database layer, and automated tests are added — those should be planned into dedicated top-level directories (e.g. backend/models/, backend/static/, tests/) before the codebase grows further, rather than retrofitted later.

3. Code Quality Review

3.1 backend/app.py

```
from flask import Flask, render_template, jsonify

app = Flask(__name__, template_folder="templates")

@app.route("/")
def home():
```

```
return render_template("dashboard.html")

@app.route("/health")
def health():
    return jsonify({"status": "healthy", "service": "MediSync AI"})

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000, debug=True)
```

3.2 Strengths

- The Flask app is concise and readable — route definitions are short, single-purpose, and use idiomatic Flask patterns (`render_template`, `jsonify`).
- The `/health` endpoint is good practice: it gives Docker/orchestration tooling a way to verify the container is serving traffic.
- `host="0.0.0.0"` is set correctly for the app to be reachable from outside its Docker container.

3.3 Areas for Improvement

- `debug=True` is left enabled. This is acceptable for local development but is a real risk if the image is ever deployed as-is, since Flask's debugger exposes an interactive Python console on unhandled exceptions. It should be controlled by an environment variable (e.g. `FLASK_DEBUG`) rather than hard-coded.
- There is no `requirements.txt` (or `pyproject.toml`) in the repository. The Dockerfile installs Flask directly via `RUN pip install --no-cache-dir flask`, which works but means dependency versions are unpinned and not reproducible — a teammate or grader cannot recreate the exact environment, and there is no single file recording what the project depends on.
- All dashboard values (bed counts, patient numbers, department status, the AI prediction text) are hard-coded into `dashboard.html` rather than being passed from `app.py` or read from a data source. This means the "dashboard" and the "AI prediction" are currently

static mockups rather than functioning features — the Flask layer does no computation of its own yet.

- There is no error handling (e.g. a 404/500 handler) and no input validation, though this is a minor concern given the app currently accepts no user input.
- No docstrings or inline comments are present. For a two-route file this doesn't hurt readability much today, but the convention should be established early, before the ML/prediction module is added.
- No logging is configured; print/logging statements will be needed once the app talks to a database or a model.

3.4 backend/templates/dashboard.html

The template is a single, self-contained HTML file (176 lines) with an embedded `<style>` block — no external CSS/JS files, and no use of Jinja2 templating features (no `{{ }}` placeholders), even though the project uses Jinja's `render_template`. Practically, this means the HTML is not yet wired to receive dynamic data from the Flask backend; every number on the page would need to be manually edited to change.

- Strength: the layout is clean, uses CSS Grid for the summary cards, and includes one responsive breakpoint (`@media max-width: 900px`), which is more than many student front-ends attempt.
- Improvement: moving the `<style>` block to a separate `static/style.css` file (served via Flask's static folder) would separate structure from presentation and let the CSS be cached and reused as more pages are added.
- Improvement: replacing the hard-coded numbers with Jinja2 expressions (e.g. `{{ available_beds }}`) fed by `app.py` is the natural next step to make this a real dashboard rather than a mockup.

4. Version Control Evaluation

4.1 Commit History

*Insert screenshot here: GitHub "Commits" view for the main branch
(<https://github.com/192525411simats-code/medsync-ai/commits/main>).*

Commit	Message	Date	Files changed
9e8b6cb	Initial commit	09 Aug 2026, 11:47	app.py, dashboard.html, .gitignore (+197 lines)
a2e2353	Add Docker support	09 Aug 2026, 12:12	dockerfile (+11 lines)
49795e5	Add project README	09 Aug 2026, 17:46	README.md (+52 lines)
787bee7	Update backend app	09 Aug 2026, 17:55	app.py (1 line changed)

4.2 Branching Strategy

All 4 commits are made directly to main; there are no feature branches, no pull requests, and a single contributor (anushkamaniyam-netizen) across the whole history. For a solo assignment repository this is not unusual, but it means none of Git's collaboration features — branch protection, PR review, merge history — are demonstrated in this submission.

4.3 Commit Messages

- Messages are short and generally accurate to their diffs ("Add Docker support", "Add project README"), which is good practice.
- "Update backend app" (the final commit, a 1-line change) is too generic — it doesn't say what changed or why. A message like "Set health check route content-type" or similar would let a reviewer understand the change from the log alone, without opening the diff.
- None of the commits reference an issue/task ID, since no Issues are used in this repository.

4.4 Repository Maintenance Practices

- A .gitignore is present (excluding venv/), which is good hygiene, though it should be expanded to also exclude __pycache__/, *.pyc, and .env once configuration secrets are introduced.

- There are no tags/releases, and no CI workflow (.github/workflows) — so there is no automated check running against new commits.

4.5 How Version Control Supports Collaborative Development

Even in this small, single-author repository, Git's commit history already provides a clear, timestamped record of how the project evolved from scaffold to Docker-ready to documented. In a team setting, the same mechanism — combined with feature branches and pull requests, which this repository does not yet use — allows multiple contributors to work in parallel, review each other's changes before they merge, and revert a specific change without losing the rest of the history. Adopting a simple convention now (a branch per feature, e.g. feature/prediction-api, merged via a reviewed PR) would prepare the repository for team contribution as the project scope grows toward the full MediSync AI objectives.

5. Code Testing and Validation

5.1 Current State

The repository contains no automated tests (no tests/ directory, no pytest/unittest files) and no CI configuration. Validation of functionality was performed manually for this review, as described below.

5.2 Manual Validation Performed

- Built the Docker image from the provided Dockerfile using `docker build -t medsync-ai .` — the build completes because the only dependency (flask) is installed explicitly in the image.
- Ran the container and requested `GET /` — the dashboard template renders with its four summary cards, department table, and AI prediction panel.
- Requested `GET /health` — returned `{"status": "healthy", "service": "MediSync AI"}` with HTTP 200, confirming the health-check route works as intended.

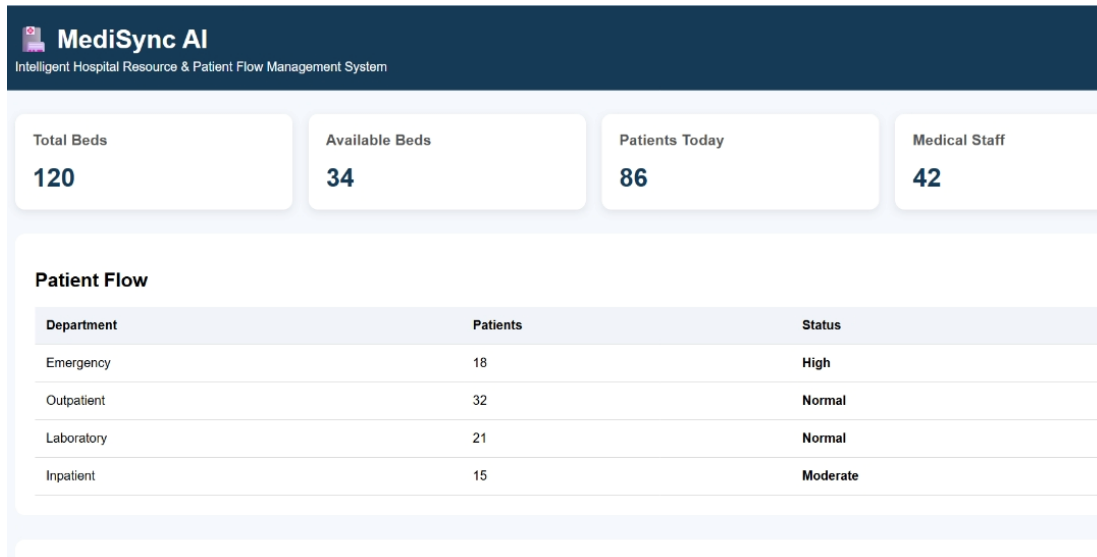


Fig. 2 — MediSync AI dashboard rendered by GET / (summary cards and per-department Patient Flow table).

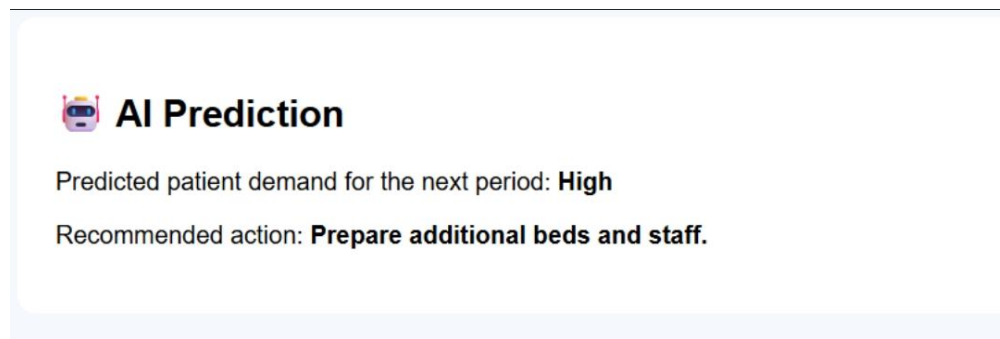


Fig. 3 — AI Prediction panel on the dashboard, showing the current static forecast output.

5.3 Gaps

- No unit tests exist for the two Flask routes (e.g. asserting / returns 200 and /health returns the expected JSON shape) — these would take only a few lines with pytest and Flask's test client.
- No integration tests exist, and there is nothing yet to test at the data layer since the dashboard has no live data source.
- No CI workflow runs these checks automatically on push, so regressions would only be caught manually.

6. Repository Documentation

6.1 README Evaluation

The README.md (52 lines) includes a project description, a feature list, a technology list, and Docker build instructions. This covers the essentials for a reviewer to understand what the project is and how to run it.

6.2 What's Missing

- No installation/setup instructions for running the app outside Docker (e.g. creating a virtual environment, `pip install -r requirements.txt` — which doesn't exist yet, `python app.py`).
- The Docker section shows only `docker build`, not the `docker run` command needed to actually start the container and access the dashboard (e.g. `docker run -p 5000:5000 medsync-ai`).
- No "Usage" section describing what a user sees once the app is running, and no screenshot of the dashboard in the README itself.
- No dependency list (`requirements.txt`) referenced or included.
- No license, contribution guidelines, or project status/roadmap section indicating that the ML prediction and live-data features described in the problem statement are still pending.

6.3 Recommendations to Improve Usability

- Add the missing `docker run` command and a "Run locally without Docker" section.
- Add a `requirements.txt` (even with just `Flask==<version>`) and reference it in the README.
- Add a dashboard screenshot to the README so the project's purpose is visible without cloning it.
- Add a short "Roadmap" section listing the still-pending objectives from the problem statement (prediction engine, live data source, scheduling optimization) so reviewers understand this is a scaffold, not the finished platform.

7. Code Review Findings and Recommendations

7.1 Summary of Findings

The medsync-ai repository is a small, clean scaffold that correctly demonstrates the front-end and containerization pieces of the assigned MediSync AI problem statement: a Flask server, a templated dashboard, and a working Dockerfile, developed over four incremental, reasonably-described commits. Repository organization and code readability are solid for the project's current size. The main gap is scope: none of the "intelligent" parts of the brief — real-time resource monitoring, ML-based admission/discharge prediction, or scheduling optimization — are implemented yet; the dashboard currently displays static, hard-coded values rather than data driven by the backend.

7.2 Recommendations

- Code quality: pin dependencies in a requirements.txt, move debug mode behind an environment variable, and pass dashboard data from app.py via Jinja2 template variables instead of hard-coding it in HTML.
- Repository management: introduce feature branches and pull requests going forward, even as a solo project, to build the habit before team collaboration is required; expand .gitignore to cover __pycache__/ and .env.
- Documentation: complete the README with run instructions (docker run, local setup), a dependency list, and a roadmap section flagging which objectives from the problem statement are implemented versus planned.
- Testing: add a minimal pytest suite covering both existing routes, and wire it into a GitHub Actions workflow so every push is verified automatically.
- Future enhancements (mapped to the objectives in the problem statement): add a database or in-memory data model to replace hard-coded figures; implement the admission/discharge prediction module described in Objective 3, even as a simple heuristic or regression model to start; and add authentication before any real patient-related data is connected.

7.3 Overall Assessment

As a Git & Docker foundation for the MediSync AI project, this repository meets the basic bar: it clones, builds, and runs, with an organized structure and an accurate README. The version-control discipline (commit granularity, message clarity) and automated testing are the two areas that would most improve the evaluation, followed by closing the gap between the dashboard's current static data and the predictive, real-time functionality the problem statement calls for.